



INSTITUTO TECNOLÓGICO
DE CIUDAD MADERO

ITCM



TECNM
TECNOLÓGICO NACIONAL DE
MÉXICO

Materia: Programacion para Big Data

Horario: 13:00 – 14:00

Docente: Jorge Peralta Escobar

Algoritmos de aprendizaje supervisado

Nombres

No. de control

Arizabalo Cepeda Erick Emmanuel 22070287

Reyna Montaña Francisco Gabriel 22070272

Periodo Escolar:

Enero – Julio 2026

El **Aprendizaje Supervisado (Supervised Learning)** es una de las ramas fundamentales del Machine Learning. En este paradigma, los algoritmos se entrenan utilizando un conjunto de datos etiquetado (*labeled data*), lo que significa que cada entrada (*input*) viene emparejada con su respectiva respuesta correcta o etiqueta (*output* o *target*).

El objetivo principal es aprender una función de mapeo f tal que $Y=f(X)$, permitiendo al modelo predecir de forma precisa la etiqueta (Y) para nuevos datos no vistos (X).

El Proceso de Desarrollo en Aprendizaje Supervisado

El desarrollo e implementación de un modelo de aprendizaje supervisado sigue un ciclo estructurado que va desde la concepción de los datos hasta el despliegue:

1. **Recolección y Etiquetado de Datos:** Es el paso crítico. Se recolectan los datos y expertos (o procesos automatizados) asignan las etiquetas correctas a cada registro.
2. **Preprocesamiento de Datos:** Limpieza de datos (manejo de valores nulos o atípicos), codificación de variables categóricas y escalado de características numéricas.
3. **División del Dataset:** El conjunto de datos se divide típicamente en:
 - o **Entrenamiento (Training set):** Para que el algoritmo aprenda los patrones (suele ser el 70-80%).
 - o **Validación/Prueba (Test set):** Para evaluar la capacidad de generalización del modelo en datos que nunca ha visto.
4. **Entrenamiento del Modelo:** El algoritmo procesa los datos de entrenamiento y ajusta sus parámetros internos mediante la optimización de una *función de pérdida* (que mide qué tan lejos está la predicción del valor real).
5. **Evaluación y Ajuste de Hiperparámetros:** Se mide el rendimiento usando métricas específicas y se modifican los ajustes externos del algoritmo (hiperparámetros) para evitar el subajuste (*underfitting*) o el sobreajuste (*overfitting*).

El aprendizaje supervisado se divide principalmente en dos grandes categorías según la naturaleza del objetivo:

- **Clasificación:** Predecir categorías discretas (ej. "Spam" o "No Spam").
- **Regresión:** Predecir valores numéricos continuos (ej. el precio de una casa).

Análisis Exhaustivo de los Algoritmos de Aprendizaje Supervisado

1. Regresión Lineal (Linear Regression)

- **¿Qué es?** Es el algoritmo base para problemas de **regresión**. Modela la relación entre una variable dependiente y una o más variables independientes ajustando una línea recta (hiperplano en dimensiones mayores).
- **Desarrollo y Funcionamiento:** Utiliza la ecuación matemática $Y = \beta_0 + \beta_1 X_1 + \dots + \beta_n X_n$. Durante el entrenamiento, calcula los coeficientes óptimos (β) minimizando la suma de los errores al cuadrado (método de Mínimos Cuadrados Ordinarios o Descenso del Gradiente).
- **Casos de uso:** Predicción de precios de viviendas basados en los metros cuadrados o estimación del peso según la altura.

2. Regresión Logística (Logistic Regression)

- **¿Qué es?** A pesar de llevar la palabra "regresión" en su nombre, es un algoritmo diseñado estrictamente para problemas de **clasificación** (generalmente binaria).
- **Desarrollo y Funcionamiento:** En lugar de trazar una línea recta infinita, toma la salida de una función lineal y la pasa a través de una función no lineal llamada **función sigmoide**. Esto comprime cualquier valor de entrada en un rango entre 0 y 1, el cual se interpreta como la probabilidad de que el dato pertenezca a una clase.
- **Casos de uso:** Detección de correos spam (0 o 1) o diagnóstico médico de presencia de una enfermedad (Sí / No).

3. Árboles de Decisión (Decision Trees)

- **¿Qué es?** Modelos predictivos en forma de estructura de árbol que pueden utilizarse tanto para clasificación como para regresión.
- **Desarrollo y Funcionamiento:** El algoritmo divide el conjunto de datos de forma jerárquica en subconjuntos cada vez más pequeños basados en reglas lógicas condicionales (ej. Si Edad > 30). Para decidir por qué característica dividir el nodo, algoritmos populares como ID3, C4.5 o CART calculan métricas de pureza como la *Ganancia de Información* o el *Índice Gini*.
- **Casos de uso:** Segmentación de clientes para otorgar créditos o sistemas de diagnóstico guiados por reglas.

4. Máquinas de Vectores de Soporte (SVM - Support Vector Machines)

- **¿Qué es?** Un algoritmo potente para clasificación y regresión que busca encontrar el límite óptimo entre las clases.
- **Desarrollo y Funcionamiento:** SVM proyecta los datos en un espacio n-dimensional y busca un hiperplano que separe las clases maximizando el "margen" (la distancia entre el hiperplano y los puntos más cercanos de cada clase, llamados *vectores de soporte*). Si los datos no son linealmente separables, utiliza el "truco del Kernel" (*Kernel Trick*) para transformar los datos a una dimensión superior donde sí se puedan separar en línea recta.
- **Casos de uso:** Clasificación de imágenes y reconocimiento de caracteres escritos a mano.

5. k-Vecinos Más Cercanos (k-NN - k-Nearest Neighbors)

- **¿Qué es?** Un algoritmo basado en instancias y de tipo "lazy learner" (aprendizaje perezoso), ya que no construye un modelo matemático explícito durante el entrenamiento.
- **Desarrollo y Funcionamiento:** Cuando llega un nuevo dato, el algoritmo calcula la distancia (usando métricas como la distancia Euclidiana o Manhattan) entre este punto y todos los datos del conjunto de entrenamiento. Luego selecciona los k vecinos más cercanos y realiza una votación mayoritaria (en clasificación) o el promedio (en regresión) para asignar el resultado.
- **Casos de uso:** Sistemas de recomendación sencillos o búsqueda de elementos similares.

6. Naive Bayes

- **¿Qué es?** Un clasificador probabilístico fundamentado en el **Teorema de Bayes**.
- **Desarrollo y Funcionamiento:** Calcula la probabilidad de que un evento ocurra dada la presencia de ciertas características. Se le llama "Naive" (ingenuo) porque asume estrictamente que todas las características de entrada son independientes entre sí, una suposición que rara vez ocurre en el mundo real pero que matemáticamente simplifica el cálculo drásticamente, haciéndolo extremadamente veloz.
- **Casos de uso:** Clasificación de texto a gran escala, análisis de sentimientos y filtrado de correo no deseado.

7. Bosques Aleatorios (Random Forest)

- **¿Qué es?** Una técnica de aprendizaje por consenso o ensamble (*Ensemble Learning*) basada en el principio de que muchos modelos débiles combinados forman un modelo robusto.
- **Desarrollo y Funcionamiento:** Crea múltiples Árboles de Decisión de forma simultánea. Para garantizar la diversidad, cada árbol se entrena con una submuestra aleatoria de los datos (*Bagging*) y un subconjunto aleatorio de las características. Para realizar una predicción, combina los resultados de todos los árboles mediante votación o promedio, lo que reduce drásticamente el sobreajuste (*overfitting*) que sufren los árboles individuales.
- **Casos de uso:** Predicción de fraude bancario o estimación del rendimiento de acciones financieras.

8. Gradient Boosting (Potenciación del Gradiente)

- **¿Qué es?** Otra técnica avanzada de ensamble que construye modelos de forma secuencial en lugar de paralela.
- **Desarrollo y Funcionamiento:** A diferencia de Random Forest, donde los árboles no interactúan, en Gradient Boosting los árboles se construyen de uno en uno. Cada nuevo árbol se diseña específicamente para corregir los errores residuales (los fallos) cometidos por los árboles anteriores empleando el descenso del gradiente. Variantes de alto rendimiento ampliamente usadas en la industria incluyen:
 - o **XGBoost:** Optimizado para velocidad y regularización.
 - o **LightGBM:** Basado en histogramas, altamente eficiente para grandes volúmenes de datos.
 - o **CatBoost:** Diseñado con un manejo nativo sobresaliente para variables categóricas.
- **Casos de uso:** Competiciones de ciencia de datos, sistemas de clasificación de motores de búsqueda y predicción de riesgo crediticio.

9. Redes Neuronales Artificiales (Artificial Neural Networks - ANN)

- **¿Qué es?** Modelos computacionales inspirados en la estructura del cerebro humano, capaces de modelar relaciones complejas y no lineales. Un ejemplo básico incluido en esta categoría es el Perceptrón Multicapa (MLP).
- **Desarrollo y Funcionamiento:** Están compuestas por capas de nodos (neuronas): una capa de entrada, una o más capas ocultas y una capa de salida. Cada conexión tiene un *peso* asociado. Durante el desarrollo, los datos fluyen hacia adelante para dar una predicción (*Forward Propagation*); luego, el error se calcula en la salida y se propaga hacia atrás (*Backpropagation*) usando el Descenso del Gradiente para actualizar los pesos de las conexiones disminuyendo progresivamente el margen de error.
- **Casos de uso:** Visión por computadora, procesamiento de lenguaje natural (NLP) y reconocimiento de voz.

Aplicacion de los Algoritmos en Practica (Codigo hecho en r)

Para desarrollar dichos algoritmos, utilizaremos el .csv de los vinos, pero primero debemos entender la naturaleza de los datos.

Este dataset contiene características químicas de muestras de vino (como acidez, azúcar, cloruros, pH, alcohol, etc.) y una variable objetivo llamada `quality` (calidad), que toma valores enteros (generalmente entre 3 y 8).

- Para los algoritmos de **Regresión**, buscaremos predecir de forma continua o numérica la calidad o el nivel de alcohol.
- Para los algoritmos de **Clasificación**, transformaremos la calidad en una variable categórica (ej. Vino "Bueno" vs "Malo") para entender cómo opera cada estructura.

Desglosando el código por partes:

```
1  # =====
2  # PREPARACIÓN INICIAL Y CARGA DE DATOS
3  # =====
4  |
5  # Instalar librerías necesarias si no se tienen
6  #install.packages(c("tidyverse", "caret", "rpart", "rpart.plot", "e1071", "class", "randomForest", "xgboost", "nnet"))
7
8  library(tidyverse)
9  library(caret)
10 library(rpart)
11 library(rpart.plot)
12 library(e1071)
13 library(class)
14 library(randomForest)
15 library(xgboost)
16 library(nnet)
17
18 # Cargar el dataset (Asegúrate de ajustar la ruta de tu archivo)
19 datos <- read.csv("Downloads/WineQT.csv")
20 # Eliminar la columna 'Id' ya que no aporta información predictiva
21 datos <- datos %>% select(-Id)
22
23 # Fijar semilla para que los resultados sean reproducibles
24 set.seed(123)
25
26 # Crear partición de entrenamiento (80%) y prueba (20%)
27 indice_entrenamiento <- createDataPartition(datos$quality, p = 0.8, list = FALSE)
28 datos_entrenamiento <- datos[indice_entrenamiento, ]
29 datos_prueba <- datos[-indice_entrenamiento, ]
30
31
```



```

30
31
32 # =====
33 # 1. REGRESIÓN LINEAL (Linear Regression)
34 # =====
35 # EN QUÉ DESTACA: Es el modelo base por excelencia para variables continuas.
36 # Destaca por su extrema simplicidad, velocidad de cómputo y alta interpretabilidad.
37 # DIFERENCIAS Y LIMITACIONES: Asume una relación estrictamente en línea recta entre
38 # las variables y la meta. Si los datos tienen curvas o interacciones complejas, falla.
39
40 modelo_lineal <- lm(quality ~ ., data = datos_entrenamiento)
41
42 # Predicción y Evaluación (Raíz del Error Cuadrático Medio - RMSE)
43 pred_lineal <- predict(modelo_lineal, newdata = datos_prueba)
44 rmse_lineal <- RMSE(pred_lineal, datos_prueba$quality)
45 cat("RMSE Regresión Lineal:", rmse_lineal, "\n")
46
47
48 # =====
49 # TRANSFORMACIÓN PARA CLASIFICACIÓN
50 # =====
51 # Para los siguientes algoritmos de clasificación, convertiremos la calidad en una
52 # variable binaria: "Malo" (calidad <= 5) y "Bueno" (calidad >= 6).
53 convertir_clase <- function(df) {
54   df %>%
55     mutate(calidad_binaria = ifelse(quality >= 6, "Bueno", "Malo")) %>%
56     mutate(calidad_binaria = as.factor(calidad_binaria)) %>%
57     select(-quality)
58 }
59
60 train_clasif <- convertir_clase(datos_entrenamiento)
61 test_clasif <- convertir_clase(datos_prueba)
62

```

```

62
63
64 # =====
65 # 2. REGRESIÓN LOGÍSTICA (Logistic Regression)
66 # =====
67 # EN QUÉ DESTACA: Es el estándar de oro para clasificación binaria cuando buscas
68 # entender el "por qué". Te da las probabilidades exactas de pertenecer a una clase.
69 # MEJORAS SOBRE R. LINEAL: En lugar de predecir una línea infinita (que daría valores
70 # imposibles como calidad de vino de -5 o 100), aplica la función sigmoide para
71 # acotar los resultados estrictamente entre 0 y 1 (probabilidad).
72
73 modelo_logistico <- glm(calidad_binaria ~ ., data = train_clasif, family = "binomial")
74
75 # Predicciones (Devuelve probabilidades, usamos umbral de 0.5)
76 prob_logistica <- predict(modelo_logistico, newdata = test_clasif, type = "response")
77 pred_logistica <- factor(ifelse(prob_logistica > 0.5, "Bueno", "Malo"), levels = c("Bueno", "Malo"))
78
79 # Evaluación
80 matriz_logistica <- confusionMatrix(pred_logistica, test_clasif$calidad_binaria)
81 cat("Precisión (Accuracy) Regresión Logística:", matriz_logistica$overall["Accuracy"], "\n")
82
83

```

```

83
84 # =====
85 # 3. ÁRBOLES DE DECISIÓN (Decision Trees)
86 # =====
87 # EN QUÉ DESTACA: Excelente para capturar reglas de negocio ("Si el alcohol es > 10.5
88 # y la acidez es baja, entonces el vino es Bueno"). No requiere que los datos estén escalados.
89 # MEJORA SOBRE LAS REGRESIONES: Puede modelar relaciones no lineales y cortes abruptos
90 # en los datos de forma nativa sin necesidad de fórmulas complejas. Es totalmente visual.
91
92 modelo_arbol <- rpart(calidad_binaria ~ ., data = train_clasif, method = "class")
93
94 # Dibujar el árbol para ver las reglas de decisión
95 rpart.plot(modelo_arbol, main = "Árbol de Decisión - Calidad del Vino")
96
97 # Predicción y Evaluación
98 pred_arbol <- predict(modelo_arbol, newdata = test_clasif, type = "class")
99 matriz_arbol <- confusionMatrix(pred_arbol, test_clasif$calidad_binaria)
100 cat("Precisión Árbol de Decisión:", matriz_arbol$overall["Accuracy"], "\n")
101
102

```

```

103 # =====
104 # 4. MÁQUINAS DE VECTORES DE SOPORTE (SVM)
105 # =====
106 # EN QUÉ DESTACA: Es sumamente efectivo en espacios de alta dimensionalidad (muchas variables).
107 # MEJORA SOBRE ÁRBOLES Y REGRESIONES: Introduce el "Truco del Kernel" (aquí usamos 'radial').
108 # Si los vinos buenos y malos están mezclados de forma compleja, SVM proyecta los datos a una
109 # dimensión matemática superior donde sí pueda trazar una línea o frontera limpia para separarlos.
110
111 modelo_svm <- svm(calidad_binaria ~ ., data = train_clasif, kernel = "radial", probability = TRUE)
112
113 # Predicción y Evaluación
114 pred_svm <- predict(modelo_svm, newdata = test_clasif)
115 matriz_svm <- confusionMatrix(pred_svm, test_clasif$calidad_binaria)
116 cat("Precisión SVM:", matriz_svm$overall["Accuracy"], "\n")
117
118

```

```

118
119 # =====
120 # 5. k-VECINOS MÁS CERCANOS (k-NN)
121 # =====
122 # EN QUÉ DESTACA: Es un algoritmo intuitivo basado en analogías ("Dime con quién andas y te
123 # diré quién eres"). No asume ninguna distribución matemática de los datos.
124 # DIFERENCIAS CLAVE: A diferencia de todos los anteriores, k-NN NO aprende reglas ni
125 # coeficientes durante el entrenamiento. Solo guarda los datos y calcula distancias geométricas.
126 # NOTA DE DESARROLLO: Requiere obligatoriamente normalizar/escalar las variables.
127
128 # Escalamos los datos numéricos (excluyendo la variable objetivo)
129 escalador <- preProcess(train_clasif[, -12], method = c("center", "scale"))
130 train_escalado <- predict(escalador, train_clasif[, -12])
131 test_escalado <- predict(escalador, test_clasif[, -12])
132
133 # Ejecución de KNN (usando k=5 vecinos como ejemplo)
134 pred_knn <- knn(train = train_escalado, test = test_escalado,
135 | | | | | | | | cl = train_clasif$calidad_binaria, k = 5)
136
137 # Evaluación
138 matriz_knn <- confusionMatrix(pred_knn, test_clasif$calidad_binaria)
139 cat("Precisión k-NN:", matriz_knn$overall["Accuracy"], "\n")
140

```



```

141
142 # =====
143 # 6. NAIVE BAYES
144 # =====
145 # EN QUÉ DESTACA: Es increíblemente veloz y requiere muy pocos datos para entrenar.
146 # Trabaja mediante probabilidades condicionales combinadas.
147 # DIFERENCIAS Y LIMITACIONES: Asume "ingenuamente" que todas las características
148 # químicas del vino son 100% independientes entre sí (ej. asume que el pH no tiene relación
149 # con la acidez), lo cual es falso, pero a pesar de eso suele dar buenos resultados basales.
150
151 modelo_nb <- naiveBayes(calidad_binaria ~ ., data = train_clasif)
152
153 # Predicción y Evaluación
154 pred_nb <- predict(modelo_nb, newdata = test_clasif)
155 matriz_nb <- confusionMatrix(pred_nb, test_clasif$calidad_binaria)
156 cat("Precisión Naive Bayes:", matriz_nb$overall["Accuracy"], "\n")
157
158

```

```

159 # =====
160 # 7. BOSQUES ALEATORIOS (Random Forest) - Ensamble en Paralelo
161 # =====
162 # EN QUÉ DESTACA: Es uno de los algoritmos más robustos y estables de la industria.
163 # MEJORAS SOBRE UN ÁRBOL ÚNICO: Un solo árbol de decisión tiende a sobreajustarse (memorizar)
164 # los datos. Random Forest soluciona esto creando cientos de árboles (ej. 500) donde cada uno
165 # ve datos y variables diferentes. Al final, hacen una votación democrática. Reduce el error drásticamente.
166
167 modelo_rf <- randomForest(calidad_binaria ~ ., data = train_clasif, ntree = 500)
168
169 # Predicción y Evaluación
170 pred_rf <- predict(modelo_rf, newdata = test_clasif)
171 matriz_rf <- confusionMatrix(pred_rf, test_clasif$calidad_binaria)
172 cat("Precisión Random Forest:", matriz_rf$overall["Accuracy"], "\n")
173

```

```

174
175 # =====
176 # 8. GRADIENT BOOSTING (XGBoost) - Ensamble Secuencial
177 # =====
178 # EN QUÉ DESTACA: Es el algoritmo rey en competencias de ciencia de datos debido a su altísima precisión.
179 # MEJORAS SOBRE RANDOM FOREST: En lugar de crear árboles independientes al mismo tiempo,
180 # XGBoost crea los árboles de uno en uno, de forma secuencial. Cada nuevo árbol se enfoca
181 # exclusivamente en corregir los errores cometidos por el árbol anterior (mediante Descenso del Gradiente).
182 # NOTA DE DESARROLLO: Requiere transformar los datos a matrices numéricas estrictas y etiquetas 0/1.
183
184 # Preparación específica para XGBoost
185 X_train <- as.matrix(train_clasif[, -12])
186 y_train <- ifelse(train_clasif$calidad_binaria == "Bueno", 1, 0)
187 X_test <- as.matrix(test_clasif[, -12])
188
189 modelo_xgb <- xgboost(data = X_train, label = y_train, nrounds = 50,
190 | | | | | | | | | | objective = "binary:logistic", verbose = 0)
191
192 # Predicción y Evaluación (Umbral 0.5)
193 prob_xgb <- predict(modelo_xgb, newdata = X_test)
194 pred_xgb <- factor(ifelse(prob_xgb > 0.5, "Bueno", "Malo"), levels = c("Bueno", "Malo"))
195
196 matriz_xgb <- confusionMatrix(pred_xgb, test_clasif$calidad_binaria)
197 cat("Precisión Gradient Boosting (XGBoost):", matriz_xgb$overall["Accuracy"], "\n")
198
199

```

